Objetivos da aula:

- Compreender o conceito de ORM e sua aplicação em sistemas Node.js
- Identificar os principais ORMs do ecossistema e suas características
- Utilizar o Drizzle ORM para realizar operações de CRUD
- Definir schema de banco de dados em TypeScript
- Gerar e aplicar migrations com Drizzle Kit
- Entender o papel das migrations na evolução do banco de dados
- Relacionar o uso de ORM com a camada de dados em arquiteturas como MVC, MVP e MVVM

Veja o vídeo para tirar dúvidas: https://youtu.be/8ovXeHHCN_4

Índice:

i. Introdução ao ORM e principais ferramentas no Node.js
ii. Introdução prática ao Drizzle ORM
iii. Migrations com Drizzle ORM

i. Introdução ao ORM e principais ferramentas no Node.js

1.1. Introdução

Nas aulas anteriores, foram estudados conceitos fundamentais de organização de sistemas, tanto no nível de **interação entre objetos** quanto na **estrutura arquitetural da aplicação**.

Esses conceitos levaram à separação clara entre:

- interface
- lógica de negócio
- acesso a dados

Entretanto, ainda permanece uma questão central no desenvolvimento de aplicações modernas:

Como a aplicação interage com o banco de dados de forma organizada, segura e desacoplada?

É nesse contexto que surge o conceito de ORM (Object-Relational Mapping).

1.2. Motivação: Problema do Acesso Direto ao Banco

Considere um sistema Node.js que acessa diretamente um banco de dados relacional utilizando SQL:

```
const result = await db.query(
  "SELECT * FROM usuarios WHERE id = $1",
  [id]
);
```

Embora funcional, esse tipo de abordagem apresenta diversos problemas:

- mistura lógica de negócio com acesso a dados

- forte acoplamento com SQL
- maior risco de erros (ex.: SQL injection)
- dificuldade de manutenção
- baixa reutilização de código

Além disso, o desenvolvedor precisa constantemente:

- mapear dados do banco → objetos
- converter objetos → comandos SQL

Esse processo é repetitivo e propenso a erros.

1.3. O que é um ORM

ORM (Object-Relational Mapping) é uma técnica que permite mapear:

- tabelas do banco de dados → classes
- linhas → objetos
- colunas → atributos

Ou seja, o ORM cria uma camada intermediária entre a aplicação e o banco de dados.

Em vez de escrever SQL diretamente, o desenvolvedor trabalha com objetos:

```
const usuario = await Usuario.findById(1);
```

Internamente, o ORM é responsável por:

- gerar SQL automaticamente
- executar queries
- mapear resultados para objetos

1.4. Benefícios do ORM

O uso de ORM traz diversas vantagens no desenvolvimento de software:

✓ Abstração do banco de dados

O desenvolvedor não precisa escrever SQL manualmente na maioria dos casos.

✓ Aumento de produtividade

Menos código repetitivo e mais foco na lógica de negócio.

✓ Redução de erros

Evita problemas comuns como SQL mal formatado ou vulnerabilidades.

✓ Integração com a linguagem

Permite trabalhar com conceitos da linguagem (classes, objetos, tipos).

✓ Melhor organização do código

Alinha-se com a separação de responsabilidades estudada anteriormente.

1.5. Limitações do ORM

Apesar das vantagens, o uso de ORM também possui limitações importantes:

- perda de controle fino sobre queries
- possível degradação de performance em consultas complexas
- curva de aprendizado do framework
- abstrações que podem esconder comportamento do banco

Por isso, em sistemas reais, é comum combinar:

- ORM → operações comuns
- SQL direto → casos específicos e otimizados

1.6. ORM no Ecossistema Node.js

No ecossistema Node.js, existem diversos ORMs e ferramentas relacionadas. Cada um possui características próprias, especialmente em relação a:

- tipagem (TypeScript)
- nível de abstração
- performance
- estilo de uso

A seguir estão os principais players.

1.7. Principais ORMs no Node.js

1.7.1. Sequelize

Um dos ORMs mais antigos e populares do Node.js.

Características:

- suporte a múltiplos bancos (PostgreSQL, MySQL, SQLite)
- API baseada em classes e modelos
- bastante maduro

Exemplo:

```
// Definição do tipo (modelo)
const Usuario = sequelize.define("Usuario", {
  id: {
    type: DataTypes.INTEGER,
    primaryKey: true,
    autoIncrement: true,
  },
  nome: DataTypes.STRING,
});
```

```
// Consulta
const usuario = await Usuario.findByPk(1);
```

✓ Vantagem: estabilidade

✗ Desvantagem: menos moderno, tipagem limitada

1.7.2. TypeORM

Muito utilizado em projetos com TypeScript.

Características:

- forte integração com TypeScript
- uso de decorators
- inspirado em ORMs de outras linguagens (como Java/Hibernate)

Exemplo: Definição da entidade que representa uma tabela.

```
// Definição do tipo (entidade)
@Entity()
class Usuario {
  @PrimaryGeneratedColumn()
  id: number;

  @Column()
  nome: string;
}

// Consulta
const usuario = await repository.findOneBy({ id: 1 });
```

✓ Vantagem: boa integração com orientação a objetos

✗ Desvantagem: complexidade e inconsistências em versões recentes

1.7.3. Prisma

Um dos ORMs modernos mais populares atualmente.

Características:

- foco em produtividade
- geração automática de cliente
- forte tipagem
- uso de schema declarativo

Exemplo:

```
// Definição do tipo (schema.prisma)
model User {
  id   Int    @id @default(autoincrement())
  nome String
```

```
}
// Consulta
const usuario = await prisma.user.findUnique({
  where: { id: 1 },
});
```

✓ Vantagem: excelente experiência de desenvolvimento

✗ Desvantagem: camada adicional (schema + geração de código)

1.7.4. Drizzle ORM

Framework moderno e leve, foco desta aula.

Características principais:

- abordagem SQL-like (mais próximo do banco)
- tipagem forte com TypeScript
- sem abstrações excessivas
- controle mais explícito das queries

Exemplo:

```
// Definição do tipo (tabela)
import { pgTable, serial, text } from "drizzle-orm/pg-core";

export const usuarios = pgTable("usuarios", {
  id: serial("id").primaryKey(),
  nome: text("nome"),
});

// Consulta
import { eq } from "drizzle-orm";

const usuario = await db
  .select()
  .from(usuarios)
  .where(eq(usuarios.id, 1));
```

✓ Vantagem: equilíbrio entre ORM e SQL

✓ Vantagem: alta performance e previsibilidade

✗ Desvantagem: menos abstração (exige mais conhecimento do banco)

1.8. Comparação Geral

ORM	Nível de abstração	Tipagem	Estilo	Indicado para
Sequelize	Alto	Baixa	Clássico	Projetos legados
TypeORM	Alto	Alta	OOP	Aplicações estruturadas

ORM	Nível de abstração	Tipagem	Estilo	Indicado para
Prisma	Médio	Muito alta	Declarativo	Alta produtividade
Drizzle	Baixo/Médio	Muito alta	SQL-like	Controle e performance

1.9. Conexão com as Aulas Anteriores

Nas aulas anteriores, foram estudados os padrões arquiteturais MVC, MVP e MVVM, cujo principal objetivo é a **separação de responsabilidades** dentro de um sistema.

Nesses padrões, a aplicação é organizada em três partes principais:

- Interface (View)
- Lógica de negócio (Controller / Presenter / ViewModel)
- Dados (Model)

O ORM atua diretamente na **camada de Model**, sendo responsável pelo acesso e manipulação dos dados persistidos.

Dentro de cada padrão, o papel do ORM pode ser entendido da seguinte forma:

- **No MVC**:
  O ORM é utilizado pelo **Model**, que encapsula a lógica de acesso ao banco. A View pode consultar o Model, mas não acessa diretamente o banco.
- **No MVP**:
  O **Presenter** utiliza o ORM por meio do Model ou de serviços, sendo responsável por buscar e preparar os dados antes de enviá-los para a View.
- **No MVVM**:
  O **ViewModel** interage com o Model (que utiliza ORM), expondo dados prontos para a View, geralmente de forma reativa.

Independentemente do padrão arquitetural adotado, o uso de ORM permite:

- isolar o acesso ao banco de dados
- reduzir o acoplamento entre camadas
- melhorar a testabilidade da aplicação

Dessa forma, o ORM não é apenas uma ferramenta técnica, mas sim um elemento que **materializa, na prática, os princípios de separação de responsabilidades** estudados anteriormente, contribuindo para sistemas mais organizados, escaláveis e fáceis de manter.

ii. Introdução prática ao Drizzle ORM

2.1. Introdução

Neste capítulo, será utilizado o projeto disponibilizado no repositório https://github.com/arleysouza/tpII-2026-1-drizzle-orm, cujo objetivo é apresentar um exemplo mínimo e didático de uso de ORM.

Diferentemente de aplicações completas, este projeto:

- não utiliza Express
- não possui interface web
- não introduz múltiplas camadas desnecessárias

O foco é compreender:

como o ORM Drizzle se integra a uma aplicação Node.js de forma direta

2.2. Ideia central do uso do ORM

No projeto, o Drizzle aparece em dois pontos principais:

- definição da tabela (schema.ts)
- execução de operações (repository.ts)

Todo o restante é apenas código TypeScript comum.

Isso permite separar claramente:

- **o que é responsabilidade do ORM**
- **o que é lógica da aplicação**

2.3. Definição do Schema

O arquivo src/schema.ts define a estrutura da tabela:

```
export const usuarios = pgTable("users", {
  id: serial("id").primaryKey(),
  nome: text("nome").notNull(),
});
```

Esse código representa:

- a tabela `users` no banco
- a coluna `id` (chave primária)
- a coluna `nome` (obrigatória)

Além disso, o Drizzle infere automaticamente os tipos TypeScript.

2.4. Conexão com o Banco

A conexão é criada no arquivo src/db.ts, que:

- lê variáveis de ambiente
- cria o pool de conexões
- exporta o objeto db

Esse objeto é utilizado em todas as operações.

2.5. Acesso aos dados (Repository)

O arquivo src/repository.ts concentra as operações de banco:

- create(nome)
- getAll()
- update(id, nome)
- remove(id)

Essa separação é importante porque:

- isola o acesso ao banco
- facilita manutenção
- prepara o terreno para arquiteturas futuras

2.6. Operações com Drizzle

Create

```
await db
    .insert(usuarios)
    .values({ nome })
    .returning();
```

Insere um registro e retorna o valor criado.

Read

```
db.select().from(usuarios).orderBy(asc(usuarios.id));
```

Retorna todos os registros ordenados.

Update

```
await db
    .update(usuarios)
    .set({ nome })
    .where(eq(usuarios.id, id))
    .returning();
```

Atualiza um registro específico.

Delete

```
await db
    .delete(usuarios)
    .where(eq(usuarios.id, id))
    .returning();
```

2.7. Execução do fluxo (demo)

O arquivo src/demo.ts executa o fluxo completo:

1. cria um usuário
2. lista os usuários
3. atualiza o registro
4. lista novamente
5. remove o registro
6. exibe o estado final

Esse fluxo permite observar o efeito de cada operação no banco de dados

2.8. Papel do ORM no projeto

Dentro do projeto, o ORM é responsável por:

- mapear a tabela (pgTable)
- executar operações (insert, select etc.)
- garantir tipagem

Enquanto isso, a aplicação continua sendo:

- funções assíncronas
- logs no console
- controle de fluxo simples

iii. Migrations com Drizzle ORM

3.1. Introdução

No projeto, a tabela **não é criada manualmente** no banco.

Ela é definida em código (schema.ts) e materializada por meio de **migrations**

Isso levanta a questão central:

como transformar o schema em estrutura real no banco?

3.2. O que são Migrations

Migrations são arquivos SQL versionados que representam alterações no banco.

Cada migration descreve uma mudança, como:

- criação de tabela
- adição de coluna
- alteração de estrutura

No projeto, esses arquivos ficam na pasta:

```
drizzle/
  0000_*.sql
  0001_*.sql
```

3.3. Como Migrations funcionam no projeto

O fluxo utilizado é:

1. alterar o schema.ts
2. gerar migration
3. aplicar no banco

3.4. Geração de Migrations

```
npm run migration:generate
```

Esse comando:

- compara o schema atual com o histórico
- gera um novo arquivo SQL

3.5. Aplicação de Migrations

```
npm run migration:runtime
```

Esse comando:

- executa as migrations pendentes
- atualiza o banco

3.6. Exemplo de alteração

Se o schema for alterado, por exemplo, incluindo o campo de e-mail:

```
export const usuarios = pgTable("users", {
  id: serial("id").primaryKey(),
  nome: text("nome").notNull(),
  email: text("email").notNull()
});
```

O Drizzle pode gerar:

```
ALTER TABLE "users" ADD COLUMN "email" text NOT NULL;
```

Depois, a migration é aplicada ao banco.

3.7. Papel da pasta meta

A pasta drizzle/meta guarda:

- histórico de snapshots
- comparação entre versões

Ela permite ao Drizzle identificar:

o que mudou entre uma versão e outra

3.8. Rollback no Drizzle

O Drizzle não utiliza migrations "down" tradicionais.

Em vez disso, o processo é:

1. ajustar novamente o schema.ts
2. gerar nova migration
3. aplicar

Ou seja:

o rollback é feito por compensação, não por reversão direta

3.9. Integração com o projeto

No contexto do projeto:

- o schema.ts define o estado desejado
- as migrations materializam esse estado
- o repository.ts usa a estrutura criada

Essa separação deixa claro:

- definição → schema
- evolução → migrations
- uso → repository

3.10. Boas práticas adotadas

No projeto, seguem-se boas práticas importantes:

- não criar tabelas manualmente
- versionar migrations
- usar .env para configuração
- manter histórico de alterações

Exercícios

Exercício 1 – Persistência de dados com Drizzle ORM na tabela

Contexto

Você deverá construir um pequeno projeto em TypeScript utilizando Drizzle ORM, PostgreSQL e migrations para persistir dados na tabela `users`.

O objetivo do exercício é consolidar os seguintes pontos estudados na aula:

- definição de **schema** com Drizzle
- geração e aplicação de **migrations**
- criação de operações básicas de persistência
- execução de um fluxo simples de CRUD em console

Diferentemente de projetos com interface web, nesta atividade o foco será exclusivamente na **camada de persistência**.

Estrutura da tabela

A persistência deverá ser feita na tabela `users`, com a seguinte estrutura lógica:

```
CREATE TABLE users (
  id_user SERIAL NOT NULL,
  name VARCHAR(100) NOT NULL,
  PRIMARY KEY(id_user)
);
```

Observação importante

Como o projeto utiliza PostgreSQL, a tabela deverá ser modelada de forma compatível com esse banco. Portanto, a coluna `id_user` deve ser tratada como chave primária autoincremental do PostgreSQL.

Exercício 2 – Persistência com relacionamento: tabela cars

Contexto

Você irá evoluir o projeto desenvolvido no Exercício 1, introduzindo um novo conceito fundamental em persistência de dados: **relacionamento entre tabelas (chave estrangeira).**

Agora, além da tabela users, o sistema deverá persistir dados na tabela cars, que possui um relacionamento com usuários.

Assim como no exercício anterior, todas as estruturas do banco devem ser criadas utilizando **migrations com Drizzle ORM**.

Estrutura das tabelas

A aplicação deverá trabalhar a tabela users do Exercício 1 e a tabela cars:

```
CREATE TABLE cars (
  id_car SERIAL NOT NULL,
  id_user INTEGER NOT NULL,
  plate CHAR(7) NOT NULL,
  PRIMARY KEY(id_car),
  FOREIGN KEY(id_user)
    REFERENCES users(id_user)
);
```

Objetivo do exercício

Você deverá:

1. definir o schema da tabela cars utilizando Drizzle
2. criar uma migration para essa nova tabela
3. aplicar a migration no banco
4. implementar operações de CRUD para cars
5. garantir a integridade do relacionamento com users